

Build an Email-Capture Landing Page

Go from zero to a live landing page — collecting real emails — using Claude Code. No coding experience needed.

You'll drive an AI agent in plain English. It writes the files; you stay in charge. By the end you'll have a real website on the internet with a working email signup — and the exact steps to do it again for any offer.

What's inside

- 1. Define your offer** — get clear before you touch a tool
- 2. Set up your tools** — the Claude app, Node, GitHub, Vercel
- 3. Start your own project** — your folder, not the course folder
- 4. Build your landing page** — with Claude Code
- 5. Add email capture** — with Kit (no API key)
- 6. Put it live** — with Vercel
- 7. Bonus: talk to any API** — the level-up
- 8. Prompt reference** — every prompt in one place
- 9. You did it** — the checklist

What you'll need

- A laptop — **macOS** or **Windows**
- An internet connection (do the setup at home, on good wifi)
- **A paid Claude plan** — Pro is the floor, from **\$20/month** (cancel anytime). The free plan does *not* include the Code tab we use all day.
- An idea for something to offer — a guide, checklist, mini-course, service. *(It can be pretend for today.)*

Everything except the Claude plan is **free**.

1. Define your offer

Before any tool, get clear on what you're building. A good landing page starts with a clear **offer**, not with code.

Answer these eight questions — write the answers down, you'll paste them into your prompts later:

1. **Who is this for?** — your audience, in plain words.
2. **What's the offer?** — what you're giving them (guide, checklist, call...).
3. **What result does it create?** — what can they do after?
4. **What's the primary call-to-action?** — join the list, get the guide, book a call.
5. **Why should they trust you?** — experience, results, background.
6. **What tone?** — pick 3 adjectives (e.g. friendly, direct, warm).
7. **What visual style?** — keep it simple (clean, minimal, earthy...).
8. **What happens after signup?** — an email? a download? a thank-you?

Then reduce it to one line:

I help **[audience]** get **[result]** through **[offer]**.

Example (Andrea, our demo): I help people stuck in a 9-to-5 get the freedom to live and work from anywhere through a step-by-step digital nomad coaching program.

Why this matters: skip this and the AI builds a generic page. The clearer your answers, the better the result. Fifteen minutes here saves hours later.

2. Set up your tools

Do this **before** the workshop, on good internet. You need four things — all free except the Claude plan. Full step-by-step for each is in the course [prep/guides](#) ([Mac](#) · [Windows](#)); here's the shape.

2.1 — The Claude desktop app

This is the tool you build with. It has a friendly **Code** tab — buttons, not a terminal — and it already includes Claude Code, so there's nothing extra to install.

- Download from claude.ai/download and sign in.
- Make sure you're on a **paid plan** (Pro, from \$20/month). When you open the **Code** tab, it should *not* ask you to upgrade.

No VS Code, no terminal. Earlier versions of this workshop used a code editor. You don't need one — the Claude app is the whole workspace.

2.2 — Node.js

One free tool that lets the website you build actually run.

- Install the **LTS** version from nodejs.org. Accept the defaults. (One current LTS — you don't need version managers.)

2.3 — Git + a GitHub account

Git is your "save points." **GitHub** is where your code lives online and where Vercel will publish it from.

- **Git:** Macs already have it. On Windows, install **Git for Windows** from git-scm.com.
- **GitHub:** create a free account at github.com.

Don't memorise Git commands — Claude Code runs them for you. You just need to know the words: **commit** (a save point) and **push** (send it to GitHub).

2.4 — A Vercel account

Where you'll publish your page to a real, live URL.

- Sign up at vercel.com — signing in *with your GitHub account* makes the deploy step one click later.

Onboarding shortcut: open the Code tab and paste the course [setup-prompt.md](#). Claude checks Node and Git and makes a test file, so you know everything works before Day 1.

3. Start your own project

Here's the golden rule that keeps you out of trouble:

The course repo is a textbook, not your workbook. You'll clone it to read the lessons and starters — but you build each project in **your own folder** that *you* own. (More: [prep/working-on-your-own-projects.md](#).)

So the very first move is to spin out your own copy. In the **Code** tab, paste:

```
Make a new folder called "my-landing-page" in my home directory – NOT inside the jungli course folder. Copy the starter files from the email-landing-page skeleton in the course repo into it. Then initialise a fresh git repo there and make a first commit ("Start my landing page"). Confirm we're now working in my-landing-page, not in the course repo.
```

Claude makes the folder, copies the starter (`CLAUDE.md` , `PROMPTS.md` , `KIT-SETUP.md`), and starts a fresh Git repo. **Everything from here happens in *your***

folder.

Why not "fork"? Forking ties your work to someone else's repo. A clean, own folder is simpler, it's yours, and it deploys to its own live URL with no fuss.

4. Build your landing page

This is the fun part. You describe your offer; Claude Code writes the whole page.

4.1 — The first build prompt

Fill in the blanks with your answers from Section 1, then paste it into the Code tab:

```
Build a responsive one-page landing site for this offer.  
Audience: [your audience]  
Offer: [your offer]  
Outcome: [the result they get]  
Primary CTA: [join list / get guide / join waitlist / book call]  
Tone: [3 adjectives]  
Style: [visual direction]  
Proof: [your bio / results / credibility]  
Sections: hero, problem, offer, benefits, about, FAQ, signup,  
footer  
Constraints: plain HTML/CSS/JS, mobile-first, simple to deploy.  
Important: leave a clear placeholder for a Kit embedded form.  
Output: ready-to-deploy files.
```

Claude generates `index.html`, `styles.css`, and maybe a `script.js`. **Read what it made** — the page and the files — and **Accept** the changes.

4.2 — Preview it

Ask Claude to show you the page:

```
Open this page in my browser so I can see it.
```

4.3 — Refine it

Your first version won't be perfect. Run these **one at a time**, previewing after each:

Tighten the copy, make the CTA clearer, and simplify the layout.

Improve the page so it feels more credible and specific without adding unnecessary sections.

Make the design feel more polished while keeping the code simple and the layout easy to scan on mobile.

4.4 — Save your progress

After each solid change, make a save point:

Please commit the current changes with a short, descriptive message.

Commit often. You can always go back to a working version — that's the whole point of save points.

5. Add email capture with Kit

Kit (formerly ConvertKit) is a mailing-list tool with a free plan. For Day 1 you'll use its **embed** — a drop-in snippet with **no API key, no secrets**.

The full click-by-click is in [KIT-SETUP.md](#). In short:

1. Sign up free at kit.com.
2. **Grow** → **Landing Pages & Forms** → **+ Create new** → **Form**. Save it.

3. Click **Embed → JavaScript tab → Copy**. You get a line like `<script async data-uid="..." src="https://you.kit.com/.../index.js"></script>` .

4. Hand it to Claude Code:

```
Integrate this Kit form embed into the signup section cleanly,
keeping the
layout simple and responsive: [paste your <script> snippet].
```

5. **Test:** submit a real email on your page, then check **Grow → Subscribers** in Kit. If it's there, it works. 🎉

No API key today. The embed is the gentlest possible path. The API method (with a secret key) is a *level-up* — see Section 7 and Build 3.

6. Put it live with Vercel

Time to publish. **Vercel** watches a GitHub repo and puts your site on the internet automatically.

6.1 — Push your project to your own GitHub repo

Let Claude walk you through it:

```
Walk me through pushing this project to a new GitHub repo under
my account, and
tell me exactly what to click at each step.
```

(First push? GitHub may ask you to sign in — a browser window opens, you log in, done. It only happens once.)

6.2 — Import to Vercel

```
Now walk me through importing this GitHub repo into Vercel so it
goes live. Tell
me exactly what to click.
```

The shape: **vercel.com** → **Add New** → **Project** → **import your repo** → **Deploy**.
No build settings to change for a plain static site. In about 30 seconds you get a live URL like `your-project.vercel.app`.

6.3 — Share it

Text your live link to a friend. That's a real website you built today.

Every future push auto-updates the live site. Change the page, commit, push — Vercel republishes on its own.

7. Bonus: talk to any API (the level-up)

Here's what most people don't realise: Claude Code isn't only for building pages. **If a service has an API, Claude Code can talk to it** — check your Kit subscribers, debug an automation, all without clicking through a dashboard.

For that you swap the no-key embed for the **Kit API**, which needs a **secret key**:

- In Kit: **Account Settings** → **Developer Settings** → create a **v4 API key**. Copy it immediately (Kit won't show it again). Requests use base `https://api.kit.com/v4` with an `X-Kit-API-Key` header — docs at developers.kit.com.

🔑 **Keep it secret.** Unlike the embed, an API key can read and change your account. **Never commit it.** Put it in a `.env` file and add `.env` to `.gitignore`. Using it inside a Claude Code chat is fine — that stays on your machine. This exact habit is the heart of **Build 3**.

The bigger idea: this works with *any* service that has an API — Stripe, Notion, GitHub, Airtable. You don't learn the API docs; you describe what you want in plain English and hand over the credentials. That's the real power of an agent: a universal interface to any tool.

8. Prompt reference

Every prompt from this guide, in one place. Copy and customise.

Start your own project

```
Make a new folder called "my-landing-page" in my home directory – NOT inside the jungli course folder. Copy the email-landing-page skeleton into it, start a fresh git repo, and confirm we're working there.
```

First build

```
Build a responsive one-page landing site for this offer.  
Audience: [...] Offer: [...] Outcome: [...] Primary CTA: [...] Tone: [3 adjectives] Style: [direction] Proof: [credibility]  
Sections: hero, problem, offer, benefits, about, FAQ, signup, footer  
Constraints: plain HTML/CSS/JS, mobile-first, simple to deploy.  
Important: leave a clear placeholder for a Kit embedded form.
```

Refine (one at a time)

```
Tighten the copy, make the CTA clearer, and simplify the layout.
```

```
Improve the page so it feels more credible and specific without adding unnecessary sections.
```

```
Make the design feel more polished while keeping the code simple and the layout easy to scan on mobile.
```

Add the Kit form

Integrate this Kit form embed into the signup section cleanly, keeping the layout simple and responsive: [paste your <script> snippet].

Save a checkpoint

Please commit the current changes with a short, descriptive message.

Go live

Walk me through pushing this to a new GitHub repo under my account and importing it into Vercel so it's live. Tell me exactly what to click at each step.

9. You did it

If you followed every step, you now have:

- ☐ A clear, defined offer
- ☐ A live landing page you built with AI
- ☐ A real email signup collecting subscribers in Kit
- ☐ Your own GitHub repo and live Vercel URL

Total cost: a paid Claude plan (from \$20/month, cancel anytime). Everything else is free.

The best part? You can repeat this whole process for any new offer, project, or idea. The prompts and the workflow are yours to keep.

Built for the Jungli AI Learning Residency · AI & Agents track.